مرور کامل پروژه‌ی Movie App

مفهوم، مثال و منطق برنامه‌نویسی همه‌ی مطالب به‌کاررفته در پروژه

React · Router · State · useEffect · API · CSS

این سند چیست و چطور بخوانمش؟

این سند **مرور کامل پروژه‌ی Movie App** است. برای هر مفهومی که در پروژه به کار رفته، سه چیز می‌خوانی:

1. **مفهوم:** این چیز چیست و چرا وجود دارد (با مثال و تشبیه ساده).
2. **کد واقعی پروژه:** همان مفهوم در کد خودت کجا استفاده شده.
3. **منطق:** برنامه قدم‌به‌قدم چه فکری می‌کند و چه اتفاقی می‌افتد.

ترتیب پیشنهادی مطالعه

1. اول بخش «نقشه‌ی کلی» را بخوان تا تصویر کلی را بدانی.
2. بعد فایل‌ها را یکی‌یکی بخوان (بخش ۳). هر فایل را کنار کد خودش در VS Code باز کن.
3. بخش‌های مفاهیم JavaScript و React و CSS برای عمیق شدن‌اند.
4. در آخر به سؤال‌های مرور جواب بده. اگر نتوانستی، به بخش مربوطه برگرد.

فهرست

- بخش ۱: نقشه‌ی کلی و جریان اجرا
- بخش ۲: مفاهیم پایه‌ی وب که پروژه به آن‌ها تکیه دارد
- بخش ۳: بررسی فایل‌به‌فایل (index.html تا api.js)
- بخش ۴: مفاهیم JavaScript که استفاده شد
- بخش ۵: مفاهیم React به تفصیل
- بخش ۶: مفاهیم CSS که استفاده شد
- بخش ۷: منطق کل برنامه به زبان الگوریتم
- بخش ۸: مرور کد (باگ‌ها و بهبودها)
- بخش ۹: سؤال‌های مرور با پاسخ
- بخش ۱۰: واژه‌نامه

بخش ۱: نقشه‌ی کلی و جریان اجرا

پروژه چه می‌کند؟

یک برنامه‌ی وب برای دیدن فیلم‌ها: صفحه‌ی اصلی فیلم‌های محبوب را از اینترنت (سایت TMDB) می‌گیرد و نشان می‌دهد، کاربر می‌تواند جستجو کند، و صفحه‌ای برای علاقه‌مندی‌ها (هنوز خالی) وجود دارد.

ساختار فایل‌ها

```
project/
├── index.html              تنها صفحه‌ی HTML
├── package.json            فهرست پکیج‌ها و اسکریپت‌ها
├── vite.config.js          تنظیمات Vite
└── src/
    ├── main.jsx            نقطه‌ی شروع React
    ├── App.jsx             چیدمان اصلی و مسیرها
    ├── components/
    │   ├── NavBar.jsx      نوار بالای صفحه
    │   └── MovieCard.jsx   کارت یک فیلم
    ├── pages/
    │   ├── Home.jsx        صفحه‌ی اصلی (لیست و جستجو)
    │   └── Favorites.jsx   صفحه‌ی علاقه‌مندی‌ها
    ├── services/
    │   └── api.js          ارتباط با TMDB
    └── css/                استایل‌ها
```

نقش هر لایه

لایه	نقش	مثال از زندگی
`index.html`	ظرف خالی	یک دیوار سفید
`main.jsx`	روشن کردن برنامه	کلید برق
`App.jsx`	اسکلت صفحه و تعیین اینکه کدام صفحه کجا بیاید	نقشه‌ی ساختمان
`pages/`	هر صفحه‌ی کامل	اتاق‌ها
`components/`	تکه‌های قابل تکرار	مبل و میز که در چند اتاق هست
`services/`	صحبت با دنیای بیرون	تلفن
`css/`	ظاهر	رنگ و دکوراسیون

جریان اجرا از لحظه‌ی باز شدن سایت

```
۱. مرورگر index.html را باز می‌کند
                ↓
۲. تگ script فایل main.jsx را اجرا می‌کند
                ↓
۳. main.jsx کامپوننت App را داخل div با "id="root می‌گذارد
                ↓
۴. BrowserRouter آدرس فعلی (URL) را می‌خواند
                ↓
۵. App.jsx اجرا می‌شود: NavBar همیشه نمایش داده می‌شود
                ↓
۶. Routes آدرس را با Routeها مقایسه می‌کند
   ├── آدرس "/"              ← Home نمایش داده می‌شود
   └── آدرس "/Favorites"     ← "favorites" نمایش داده می‌شود
                ↓
۷. Home بعد از اولین نمایش، فیلم‌ها را از API می‌گیرد
                ↓
۸. state عوض می‌شود ← React صفحه را دوباره می‌سازد ← کارت‌ها دیده می‌شوند
```

بخش ۲: مفاهیم پایه‌ی وب که پروژه به آن‌ها تکیه دارد

۱. HTML، CSS و JavaScript

- **HTML:** اسکلت و ساختار (چه چیزی در صفحه هست).
- **CSS:** ظاهر (رنگ، اندازه، چیدمان).
- **JavaScript:** رفتار (وقتی کاربر کلیک کرد چه شود).

۲. DOM

وقتی مرورگر HTML را می‌خواند، یک **درخت از عنصرها** در حافظه می‌سازد که DOM نام دارد. هر تگ یک گره (node) در این درخت است. تغییر دادن DOM یعنی تغییر دادن چیزی که روی صفحه می‌بینی.

مشکل بدون React: برای تغییر صفحه باید دستی بنویسی «این عنصر را پیدا کن و متنش را عوض کن». وقتی صفحه بزرگ شود، مدیریت این کارها بسیار سخت می‌شود.

راه‌حل React: تو فقط **توصیف** می‌کنی صفحه با این داده چه شکلی باشد؛ React خودش DOM را به‌روز می‌کند.

۳. SPA (برنامه‌ی تک‌صفحه‌ای)

پروژه‌ی تو فقط **یک** فایل HTML دارد. وقتی از Home به Favorites می‌روی، مرورگر صفحه‌ی جدید از سرور نمی‌گیرد؛ JavaScript فقط محتوای وسط را عوض می‌کند. نتیجه: سریع و بدون چشمک زدن.

تشبیه: به جای اینکه برای هر اتاق یک ساختمان جدا بسازی، یک ساختمان داری و فقط وسایل داخل اتاق را عوض می‌کنی.

۴. ماژول‌ها (import و export)

JavaScript مدرن اجازه می‌دهد کد را در چند فایل بنویسی و به هم وصل کنی:

```
// api.js: صادر می‌کند
export const getPopularMovies = async () => { ... };

// Home.jsx: وارد می‌کند
import { getPopularMovies } from "../services/api";
```

- **export نام‌دار (named):** با آکولاد وارد می‌شود و اسم باید **دقیقاً** یکی باشد (مثل `getPopularMovies`). غلط املایی همان‌جاست که خطا می‌دهد.
- **export پیش‌فرض (default):** هر فایل فقط یکی دارد و هنگام import اسم دلخواه می‌گیرد: `import Home from "./pages/Home"`.

۵. npm، Vite و باندلر

- **npm:** فروشگاه پکیج‌ها. `npm install react-router-dom` یعنی «این کتابخانه را برایم بیاور».

- **Vite:** ابزاری که (۱) سرور توسعه می‌دهد، (۲) فایل‌های JSX و CSS را به چیزی که مرورگر می‌فهمد تبدیل می‌کند، و (۳) در `npm run build` همه‌چیز را فشرده می‌کند.
- **JSX:** مرورگر JSX نمی‌فهمد. Vite آن را به JavaScript معمولی تبدیل می‌کند.

۶. HTTP و API

API یعنی یک در برای صحبت برنامه‌ها با هم. برنامه‌ی تو یک **درخواست** به سایت TMDB می‌فرستد و TMDB یک **پاسخ** (به شکل JSON) برمی‌گرداند.

```
GET https://api.themoviedb.org/3/movie/popular?api_key=XXXX
```

- `GET` : «این اطلاعات را بده».
- `https://api.themoviedb.org/3` : آدرس پایه (BASE_URL).
- `/movie/popular` : مسیر (چه چیزی می‌خواهم).
- `?api_key=XXXX` : پارامتر (هویت من).
- `&query=matrix` : پارامترهای بعدی با `&` جدا می‌شوند.

پاسخ‌ها یک **کد وضعیت** دارند: `200` یعنی موفق، `401` یعنی کلید اشتباه، `404` یعنی پیدا نشد، `500` یعنی خطای سرور.

بخش ۳: بررسی فایل‌به‌فایل

۳-۱: index.html

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

- `div#root` : ظرف خالی. همه‌ی برنامه‌ی React داخل همین یک عنصر ساخته می‌شود.
- `type="module"` : به مرورگر می‌گوید این فایل از `import` و `export` استفاده می‌کند.
- Vite اجازه می‌دهد مستقیم `main.jsx` را (حتی با JSX) بنویسی.

۳-۲: main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter> <App /> </BrowserRouter>
  </StrictMode>,
)
```

منطق خط به خط

1. `document.getElementById('root')` عنصر `div` خالی را پیدا می‌کند.
2. `createRoot(...)` به React می‌گوید «از این‌جا به بعد، مسئول این عنصر هستی».
3. `.render(...)` می‌گوید «این درخت کامپوننت‌ها را داخلش بساز».

لایه‌های تو در تو (مثل پوست پیاز)

```
StrictMode          ← کمک به پیدا کردن خطاها در حالت توسعه
  BrowserRouter     ← مسیریابی برای همه‌ی زیرمجموعه‌ها
    App             ← برنامه‌ی ما
```

چرا `BrowserRouter` بیرون از `App` است؟ هر کامپوننتی که از `Link` ، `Routes` یا `useParams` استفاده می‌کند، باید **داخل** `BrowserRouter` باشد؛ وگرنه نمی‌داند آدرس فعلی چیست و خطا می‌دهد. چون `App` از `Routes` استفاده می‌کند، باید داخل BrowserRouter باشد.

StrictMode چه می‌کند؟ فقط در حالت توسعه: هر `useEffect` را دو بار اجرا می‌کند و هشدارهای اضافه می‌دهد تا کدهای ناپاک را بگیرد. در نسخه‌ی نهایی (build) هیچ تأثیری ندارد. برای همین ممکن است در dev درخواست API را دو بار ببینی.

`import './css/index.css'` : استایل‌های سراسری را وارد می‌کند. در Vite هر `import` از یک CSS یعنی «این استایل را به صفحه اضافه کن» و **تمام CSSها سراسری‌اند** (به بخش ۶ نگاه کن).

۳-۳: App.jsx

```
function App() {
  return (
    <div>
      <NavBar />
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/favorites" element={<Favorites />} />
        </Routes>
      </main>
    </div>
  );
}
```

مفهوم: ترکیب کامپوننت‌ها (Composition)

`App` فقط تکه‌ها را کنار هم می‌چیند. نه ظاهر کارت را می‌داند، نه از API چیزی می‌داند. هر کامپوننت یک وظیفه دارد (Single Responsibility).

منطق Routes و Route

`Routes` مثل یک **فهرست قانون‌ها** است. هر `Route` می‌گوید «اگر آدرس این بود، این کامپوننت را نشان بده».

```
آدرس فعلی مرورگر: /favorites
    ← با path="/" (باشد "/" فقط وقتی دقیقاً)  خیر          مطابقت دارد؟
    ← با path="/favorites" بله  مطابقت دارد؟ ← Favorites نمایش داده شود
```

- `element={<Home />}` : عنصری که باید رندر شود. دقت کن خودِ **عنصر** (`<Home />`) را می‌دهیم، نه اسم تابع.
- `NavBar` **بیرون** از `Routes` است؛ پس در همه‌ی صفحه‌ها ثابت می‌ماند. فقط `main` عوض می‌شود.

چرا `<main>` ؟

تگ معنایی برای «محتوای اصلی صفحه». فقط یکی در هر صفحه باید باشد. برای screen readerها و سئو مفید است.

چرا `MovieCard` را Route نکردیم؟

`MovieCard` یک **کامپوننت** است که باید `movie` را به عنوان prop بگیرد. Route هیچ propی نمی‌دهد، پس `movie` می‌شد `undefined` و برنامه کرش می‌کرد. صفحه (Page) با کامپوننت فرق دارد.

۳-۴: NavBar.jsx

```
<nav className="navbar">
  <div className="navbar-brand"><Link to="/">Movie App</Link></div>
  <div className="navbar-links">
    <Link to="/" className="nav-link">Home</Link>
    <Link to="/favorites" className="nav-link">{" "}Favorites{" "}</Link>
  </div>
</nav>
```

Link در برابر a

تگ معمولی `<a href="/favorites">` مرورگر را وادار می‌کند **کل صفحه را از نو** بارگیری کند (فلاش سفید، از دست رفتن state). ولی `Link` کلیک را می‌گیرد، آدرس را با **History API** عوض می‌کند و فقط React محتوای مناسب را نشان می‌دهد. `Link` در نهایت یک `<a>` واقعی می‌سازد (برای دسترس‌پذیری و کلیک راست) ولی رفتارش را کنترل می‌کند.

`{" "}` یعنی چه؟

JSX فاصله‌های ابتدا و انتهای خط را حذف می‌کند. `{" "}` یک فاصله‌ی واقعی در متن می‌گذارد. این‌ها را Prettier خودکار اضافه کرده؛ اینجا لازم نیستند و می‌توانی حذفشان کنی.

```
import React from "react"
```

در نسخه‌های جدید React و Vite لازم نیست (JSX خودش کار می‌کند). فقط یک import اضافی است.

۳-۵: Home.jsx (مهم‌ترین فایل)

چهار state

```
const [searchQuery, setSearchQuery] = useState("");  // متن داخل کادر جستجو
const [movies, setMovies] = useState([]);            // لیست فیلم‌ها
const [error, setError] = useState(null);            // پیام خطا
const [loading, setLoading] = useState(true);        // آیا در حال بارگذاری‌ایم؟
```

چرا این چهار تا؟ هر چیزی که **با تغییرش صفحه باید عوض شود** و **کاربر یا API آن را تغییر می‌دهد**، state است:

state	چه چیزی را کنترل می‌کند
`searchQuery`	چیزی که کاربر تایپ کرده در input دیده شود
`movies`	کارت‌هایی که نمایش داده می‌شوند
`error`	نمایش یا مخفی بودن پیام خطا
`loading`	نمایش «Loading...» به جای لیست

`loading` از اول `true` است چون صفحه از لحظه‌ی اول منتظر داده است.

بارگذاری اولیه با useEffect

```
useEffect(() => {
  const loadPopularMovies = async () => {
    try {
      const popularMovies = await getPopularMovies();
      setMovies(popularMovies);
    } catch (err) {
      console.log(err);
      setError("Failed to load Movies...");
    } finally {
      setLoading(false);
    }
  };
  loadPopularMovies();
}, []);
```

منطق قدم‌به‌قدم (خط زمان)

```
زمان ۰: Home اولین بار رندر می‌شود
        movies=[]  loading=true  error=null
        ← صفحه: کادر جستجو + Loading

زمان ۱: React بعد از نمایش صفحه، useEffect را اجرا می‌کند
        تابع loadPopularMovies صدا زده می‌شود

زمان ۲: await getPopularMovies  ← درخواست به اینترنت می‌رود
        (در این مدت کاربر Loading می‌بیند و برنامه قفل نیست)

زمان ۳ (موفق): setMovies ← داده ذخیره می‌شود
                finally: setLoading(false)
        ← React دوباره رندر می‌کند ← کارت‌ها نمایش داده می‌شوند

زمان ۳ (خطا): await خطا پرتاب می‌کند ← به catch می‌رود
               setError("Failed...")  و  finally: setLoading(false)
        ← پیام خطا نمایش داده می‌شود
```

چرا `[]` در انتها؟

آرایه‌ی وابستگی (dependency array). `[]` یعنی «این effect فقط **یک بار** بعد از اولین نمایش اجرا شود». اگر آن را نگذاری، بعد از هر رندر اجرا می‌شود، و چون خود effect state را عوض می‌کند (که رندر جدید می‌سازد)، **حلقه‌ی بی‌نهایت** پیش می‌آید.

چرا تابع داخل effect تعریف شده و بعد صدا زده می‌شود؟

تابعی که به `useEffect` می‌دهی نباید `async` باشد (چون باید یا هیچ چیز یا تابع پاک‌سازی برگرداند، نه Promise). پس یک تابع async داخلش تعریف می‌کنیم و **صدا می‌زنیم** (`loadPopularMovies()`). اگر صدایش نزنی، هرگز اجرا نمی‌شود.

try / catch / finally

```
try      ← کدی که ممکن است خطا بدهد (درخواست اینترنت)
catch    ← اگر خطا آمد، اینجا مدیریتش کن (نمایش پیام)
finally  ← همیشه اجرا شود چه موفق چه ناموفق، (loading خاموش کردن)
```

اگر `setLoading(false)` فقط در `try` بود و خطا می‌آمد، «Loading...» تا ابد می‌ماند. به همین دلیل در `finally` است.

جستجو: handleSearch

```
const handleSearch = async (e) => {
  e.preventDefault();
  if (!searchQuery.trim()) return;
  if (loading) return;
  setLoading(true);
  try {
    const searchResults = await searchMovies(searchQuery);
    setMovies(searchResults);
    setError(null);
  } catch (err) {
    console.log(err);
    setError("Failed to search movies...");
  } finally {
    setLoading(false);
  }
};
```

منطق (الگوریتم)

```
۱. جلوی رفتار پیش‌فرض فرم را بگیر (جلوگیری از رفرش صفحه)
۲. اگر متن خالی است (یا فقط فاصله) ← هیچ کاری نکن و خارج شو
۳. اگر یک درخواست دیگر در جریان است ← هیچ کاری نکن و خارج شو
۴. loading را true کن
۵. نتیجه‌ی جستجو را از API بگیر
۶. نتیجه را در movies بگذار و خطای قبلی را پاک کن
۷. اگر خطا شد، پیام خطا بگذار
۸. در هر صورت، loading را false کن
```

نکته‌ها:

- `e.preventDefault()`: مرورگر به طور پیش‌فرض بعد از ارسال فرم صفحه را reload می‌کند. با این خط جلویش را می‌گیریم تا state از بین نرود.
- `.trim()`: فاصله‌های ابتدا و انتها را حذف می‌کند. `" "` بعد از trim می‌شود `""` که در شرط، **falsy** است و `!""` برابر `true` است.

- `return` **زودهنگام (Guard Clause):** به جای تودرتو کردن همه‌چیز در `if`، شرط‌های اشتباه را اول رد می‌کنیم و ادامه کد تمیز می‌ماند.
- `if (loading) return` **:** جلوی کلیک پشت‌سرهم و ارسال چند درخواست هم‌زمان را می‌گیرد.

فرم و input کنترل‌شده

```
<form onSubmit={handleSearch} className="search-form">
  <input
    value={searchQuery}
    onChange={(e) => setSearchQuery(e.target.value)}
  />
  <button type="submit">search</button>
</form>
```

Controlled Component: مقدار input همیشه از state می‌آید (`value`) و هر بار که کاربر تایپ می‌کند، state به‌روز می‌شود (`onChange`).

```
کاربر حرف "m" می‌زند
  ← onChange اجرا می‌شود (e.target.value = "m")
  ← setSearchQuery("m")
  ← React Home را دوباره رندر می‌کند
  ← input با value="m" نمایش داده می‌شود
```

نتیجه: **state منبع حقیقت (Single Source of Truth) است**، نه خود input. هر وقت بخواهی مقدار تایپ‌شده را بدانی، `searchQuery` را بخوان.

چرا `onSubmit` روی form، نه `onClick` روی دکمه؟ چون با `onSubmit` هم کلیک روی دکمه و هم زدن Enter در کادر کار می‌کند.

رندر شرطی

```
{error && <div className="error-message">{error}</div>}

{loading ? (
  <div className="loading">Loading...</div>
) : (
  <div className="movies-grid">
    {movies.map((movie) => (
      <MovieCard movie={movie} key={movie.id} />
    ))}
  </div>
)}
```

- `error && <div>` **:** اگر `error` برابر `null` باشد، `null && ...` همان `null` را برمی‌گرداند و React برای `null` چیزی نمایش نمی‌دهد. اگر رشته‌ی پیام باشد، `div` نمایش داده می‌شود.
- `loading ? A : B` **:** عملگر سه‌تایی. اگر loading درست بود A، وگرنه B.

- `movies.map(...)`: از هر آیتم آرایه یک `MovieCard` می‌سازد. نتیجه یک آرایه از عنصرهاست که React نمایش می‌دهد.
- `key={movie.id}`: هر عنصر لیست باید یک شناسه‌ی یکتا داشته باشد تا React هنگام تغییر لیست بداند کدام عنصر عوض شده، کدام اضافه و کدام حذف شده است. اگر نداشته باشد، کارایی پایین می‌آید و ممکن است state عنصرها به هم بریزد.

۳-۶: MovieCard.jsx

```
function MovieCard({ movie }) {
  function onFavoriteClick() { alert("clicked"); }
  return (
    <div className="movie-card">
      <div className="movie-poster">
        <img src={`https://image.tmdb.org/t/p/w500${movie.poster_path}`} alt={movie.title} />
        <div className="movie-overlay">
          <button className="favorite-btn" onClick={onFavoriteClick}>♥</button>
        </div>
      </div>
      <div className="movie-info">
        <h3>{movie.title}</h3>
        <p>{movie.release_date?.split("-")[0]}</p>
      </div>
    </div>
  );
}
```

props و Destructuring

`function MovieCard({ movie })` یعنی «کامپوننت یک آبجکت props می‌گیرد؛ من فقط `movie` را از آن بیرون می‌کشم». معادل:

```
function MovieCard(props) {
  const movie = props.movie;
}
```

در `Home` با `<MovieCard movie={movie} />` این prop داده می‌شود.

تشبیه: کامپوننت یک **قالب کلوچه** است و props **خمیر**. یک قالب، بسته به خمیر، کلوچه‌های متفاوت می‌سازد.

آدرس عکس با Template Literal

```
`https://image.tmdb.org/t/p/w500${movie.poster_path}`
```

بین بک‌تیک‌ها، `${...}` یک مقدار جاوااسکریپت را داخل رشته می‌گذارد. API فقط **بخش آخر آدرس** (مثل `/abc123.jpg`) را برمی‌گرداند و آدرس پایه (`https://image.tmdb.org/t/p/w500`) را خودمان جلویش می‌گذاریم. `w500` عرض ۵۰۰ پیکسل است.

Optional Chaining و تاریخ

```
movie.release_date?.split("-")[0]
```

`release_date` مثلاً `"2020-10-22"` است. `split("-")` آن را به آرایه‌ی `["2020","10","22"]` تبدیل می‌کند و `[0]` سال را برمی‌دارد. علامت `?.` یعنی «اگر `release_date` وجود نداشت (`undefined`)، خطا نده و نتیجه را `undefined` کن». بدون آن، برای فیلمی که تاریخ ندارد، برنامه کرش می‌کرد.

رویداد کلیک

`onClick={onFavoriteClick}` یعنی «وقتی کلیک شد این تابع را صدا بزن». دقت کن تابع را **پاس می‌دهیم**، نه **صدا می‌زنیم**. `onClick={onFavoriteClick()}` اشتباه است (در همان لحظه‌ی رندر اجرا می‌شود).

فعلاً `alert("clicked")` یک جای‌نگهدار است؛ در مرحله‌ی بعد با Context به لیست علاقه‌مندی‌ها وصل می‌شود.

نکته: فیلم بدون پوستر

اگر `poster_path` برابر `null` باشد، آدرس عکس می‌شود `.../w500null` و عکس شکسته نمایش داده می‌شود. بهتر است شرط بگذاری: «اگر پوستر داشت، عکس؛ وگرنه یک تصویر جایگزین».

۳-۷: Favorites.jsx

یک صفحه‌ی ایستا که حالت **خالی (Empty State)** را نشان می‌دهد. هر برنامه‌ی خوب سه حالت را در نظر می‌گیرد: **بارگذاری، خطا، خالی**. این صفحه به‌زودی با داده‌ی واقعی (از Context) پر می‌شود.

۳-۸: api.js

```
const API_Key = "..."
const BASE_URL = "https://api.themoviedb.org/3"

export const getPopularMovies = async () => {
  const response = await fetch(`${BASE_URL}/movie/popular?api_key=${API_Key}`)
  const data = await response.json()
  return data.results
}

export const searchMovies = async (query) => {
  const response = await fetch(`${BASE_URL}/search/movie?
api_key=${API_Key}&query=${encodeURIComponent(query)}`)
  const data = await response.json()
  return data.results
}
```

چرا فایل جدا (services)؟

جداسازی مسئولیت‌ها: کامپوننت‌ها فقط می‌گویند «فیلم‌های محبوب را بده». اینکه آدرس چیست و کلید چیست، مسئله‌ی `api.js` است. اگر روزی API عوض شد، فقط یک فایل را تغییر می‌دهی.

async / await و Promise

درخواست اینترنت زمان‌بر است. جاوااسکریپت منتظر نمی‌ماند؛ به جای نتیجه یک **Promise** (قول) می‌دهد که بعداً پر می‌شود.

- `async` جلوی تابع: تابع همیشه یک Promise برمی‌گرداند.
- `await` : «همین‌جا صبر کن تا قول عملی شود، ولی بقیه‌ی برنامه (مثل کلیک کاربر) قفل نشود».

تشبیه: سفارش غذا می‌دهی و رسید (Promise) می‌گیری. تا غذا بیاید می‌توانی کار دیگری بکنی. `await` یعنی «این خط تا آمدن غذا جلوتر نرود».

دو مرحله‌ی fetch

```
const response = await fetch(url);    // مرحله‌ی ۱: هدر پاسخ رسید
const data = await response.json();   // مرحله‌ی ۲: بدنه‌ی پاسخ خوانده و به آبجکت تبدیل شد
```

پاسخ TMDB چیزی شبیه این است:

```
{ "page": 1, "results": [ { "id": 1, "title": "...", "poster_path": "/x.jpg" } ] }
```

برای همین `data.results` را برمی‌گردانیم (فقط آرایه‌ی فیلم‌ها).

encodeURIComponent

آدرس اینترنتی نمی‌تواند همه‌ی کاراکترها را داشته باشد. این تابع کاراکترهای خاص را امن می‌کند:

```
"star wars & more"  ←  "star%20wars%20%26%20more"
```

بدون آن، یک `&` در متن جستجو، پارامتر جدید فرض می‌شود و جستجو خراب می‌شود.

export نام‌دار

`export const getPopularMovies` یعنی از این فایل با اسم `getPopularMovies` قابل import است. اگر در `Home.jsx` اسمش را کمی متفاوت بنویسی (مثل Populer)، import شکست می‌خورد.

بخش ۴: مفاهیم JavaScript که استفاده شد

مفهوم	مثال در پروژه	توضیح
`const`	`const BASE_URL = "..."`	متغیری که دوباره مقدار نمی‌گیرد. پیش‌فرض را `const` بگذار
Arrow function	`() => {}`	شکل کوتاه تابع
Template literal	`${BASE_URL}` داخل بک‌تیک	رشته با مقدار داخلش
Destructuring	`{ movie }` و `[a, setA]`	بیرون کشیدن مقدار از آبجکت یا آرایه
Optional chaining	`x?.split()`	خطا نده اگر `x` وجود ندارد
`map`	`movies.map(...)`	از هر آیتم یک چیز جدید می‌سازد
`trim`	`searchQuery.trim()`	حذف فاصله‌ی ابتدا و انتها
Truthy / Falsy	`!searchQuery.trim()`	مقدارهایی که در شرط false حساب می‌شوند
`&&` و `? :`	`error && ...`	شرط در JSX
async / await	`await fetch()`	کار با عملیات زمان‌بر
try/catch/finally	در `Home.jsx`	مدیریت خطا
Modules	`import` و `export`	تقسیم کد بین فایل‌ها

Destructuring با مثال

```
// آبجکت
const movie = { id: 1, title: "Matrix" };
const { title } = movie;      // title = "Matrix"

// آرایه
const pair = ["hello", () => {}];
const [value, setter] = pair; // value = "hello"
```

`useState` یک آرایه‌ی دوتایی برمی‌گرداند و `const [movies, setMovies] = useState([])` همین را باز می‌کند. اسم‌ها دلخواه‌اند ولی قرارداد `x` و `setX` است.

Falsy ها

در شرط‌ها این مقدارها `false` حساب می‌شوند: `false` ، `0` ، `""` ، `null` ، `undefined` ، `NaN` . بقیه (از جمله `"0"` و `[]`) true هستند.

map، filter و ...

```
const nums = [1, 2, 3];
nums.map((n) => n * 2);       // [2, 4, 6]   تبدیل هر آیتم
nums.filter((n) => n > 1);    // [2, 3]      نگه داشتن بعضی‌ها
nums.some((n) => n === 2);    // true        آیا حداقل یکی شرط را دارد؟
```

این‌ها هرگز آرایه‌ی اصلی را تغییر نمی‌دهند و آرایه‌ی **جدید** برمی‌گردانند. این موضوع برای React حیاتی است.

Promise و حالت‌ها

یک Promise سه حالت دارد: **pending** (در انتظار)، **fulfilled** (موفق)، **rejected** (خطا). `await` منتظر موفق شدن می‌ماند و اگر rejected شد، خطا پرتاب می‌کند تا `catch` بگیردش.

بخش ۵: مفاهیم React به تفصیل

۱. کامپوننت

یک **تابع جاوااسکریپت** که JSX برمی‌گرداند و اسمش با حرف بزرگ شروع می‌شود. کامپوننت = یک تکه‌ی مستقل از رابط کاربری.

۲. JSX

ظاهر آن شبیه HTML است ولی در واقع جاوااسکریپت است. قوانین مهم:

1. فقط یک والد: چند عنصر هم‌سطح را در یک `div` یا `<>...</>` بگذار.
2. `className` به جای `class`.
3. همه‌ی تگ‌ها بسته شوند (`<img />`).
4. جاوااسکریپت داخل `{ }`.
5. ویژگی‌ها camelCase: `onClick`، `onChange`.

۳. Props

ورودی‌های کامپوننت، مثل آرگومان‌های تابع. **فقط خواندنی‌اند** و از والد به فرزند می‌روند.

۴. State

حافظه‌ی کامپوننت. وقتی state عوض می‌شود، React کامپوننت را **دوباره رندر** می‌کند.

قانون‌های state:

1. همیشه با تابع set عوض شود (`setMovies(...)`)، هرگز مستقیم (`movies = ...`).
2. نسخه‌ی **جدید** بساز، نسخه‌ی قبلی را دستکاری نکن (Immutability).
3. set کردن بلافاصله مقدار را عوض نمی‌کند؛ مقدار جدید در **رندر بعدی** دیده می‌شود.

```
setLoading(true);
console.log(loading);  // هنوز مقدار قبلی را می‌بینی!
```

۵. چرخه‌ی رندر

```
۱. مرحله‌ی React Render:  تابع کامپوننت را اجرا می‌کند و JSX جدید می‌گیرد
۲. مرحله‌ی React Commit:  تفاوت JSX جدید و قبلی را پیدا می‌کند و فقط همان قسمت DOM را تغییر می‌دهد
۳. مرحله‌ی Effect:  بعد از نمایش روی صفحه، useEffectها اجرا می‌شوند
```

اینکه React فقط **تفاوت** را در DOM اعمال می‌کند به Virtual DOM و Reconciliation معروف است و دلیل سرعت آن است.

۶. useEffect

برای **کار جانبی** که به رندر مربوط نیست: درخواست شبکه، تایمر، خواندن localStorage.

```
useEffect(() => {
  // کار
}, [وابستگی‌ها]);
```

وابستگی	زمان اجرا
`[]`	فقط بعد از اولین نمایش
`[x]`	اولین بار و هر وقت `x` عوض شود
بدون آرایه	بعد از هر رندر

اصل مهم: داخل بدنه‌ی کامپوننت (بیرون از effect و handler) هیچ کار جانبی (مثل fetch یا setState) نکن؛ چون در هر رندر اجرا می‌شود و حلقه‌ی بی‌نهایت می‌سازد.

۷. جریان داده‌ی یک‌طرفه

```
Home (state: movies)
  |  props: movie
  ▼
MovieCard (فقط نمایش می‌دهد)
```

داده از بالا به پایین می‌آید. فرزند نمی‌تواند state والد را مستقیم تغییر دهد؛ اگر لازم بود، والد یک **تابع** به‌عنوان prop می‌دهد و فرزند آن را صدا می‌زند (مثلاً بعداً `onFavoriteClick`).

۸. رندر لیست و key

هر عنصر `map` شده، یک `key` یکتا و ثابت می‌خواهد (`movie.id`). از index آرایه استفاده نکن، مگر لیست هرگز عوض نشود.

۹. رندر شرطی

سه الگو:

```
{cond && <A />}            // اگر شرط درست بود نشان بده
{cond ? <A /> : <B />}     // یکی از دو
{list.length === 0 && ...} // حالت خالی
```

دقت: `{0 && <A />}` عدد `0` را روی صفحه چاپ می‌کند؛ برای این حالت از `count > 0 &&` استفاده کن.

۱۰. فرم‌های کنترل‌شده

مقدار input را `value` از state می‌گیرد و `onChange` آن را به‌روز می‌کند. مزیت: هر لحظه به مقدار و اعتبارسنجی دسترسی داری.

۱۱. مسیریابی کلاینتی

`BrowserRouter` با **History API** مرورگر آدرس را بدون بارگیری صفحه عوض می‌کند و `Routes` بر اساس آدرس کامپوننت مناسب را نشان می‌دهد. `Link` همان `a` است ولی بدون reload.

۱۲. StrictMode

در حالت توسعه، effectها را دو بار اجرا و هشدار اضافه می‌دهد تا مشکلات پنهان (مثل نبودن پاک‌سازی) پیدا شود. در build نهایی اثری ندارد.

بخش ۶: مفاهیم CSS که استفاده شد

۱. CSS سراسری است

در این پروژه هر فایل CSS (حتی اگر در یک کامپوننت import شود) روی **کل صفحه** اثر می‌گذارد. مثال: `.movies-grid >` در `Favorites.css` تعریف شده ولی چون `Home.css` هم `.movies-grid` دارد، هر دو اثر می‌کنند. برای همین اسم `*` کلاس‌ها باید منحصربه‌فرد و دقیق باشند و باید **دقیقاً** با `className` در JSX یکی باشند (مثلاً `movies-grid` با `s`).

۲. Reset و box-sizing

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

- فاصله‌های پیش‌فرض مرورگر را صفر می‌کند تا رفتارها یکدست باشد.
- `border-box` : عرض و ارتفاع **شامل** padding و border می‌شوند؛ پس محاسبه‌ی اندازه ساده‌تر است.

۳. Flexbox (چیدمان یک‌بعدی)

```
.navbar { display: flex; justify-content: space-between; align-items: center; }
```

- `display: flex` : فرزندها در یک ردیف.
- `justify-content` : توزیع در **محور اصلی** (افقی): `space-between` یعنی اولی چپ و آخری راست.
- `align-items` : توزیع در **محور فرعی** (عمودی): `center` یعنی وسط.
- `gap` : فاصله‌ی بین آیتم‌ها.

۴. ترکیب flex برای پر کردن ارتفاع صفحه

```
#root { min-height: 100vh; display: flex; flex-direction: column; }
.main-content { flex: 1; }
```

`#root` ستونی است و حداقل ارتفاعش تمام صفحه است. `flex: 1` به `main` می‌گوید «تمام فضای باقی‌مانده را بگیر». نتیجه: محتوا همیشه تا پایین صفحه کشیده می‌شود.

۵. Grid (چیدمان دوبعدی)

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 1.5rem;
}
```

بخوانش: «هر ستون حداقل ۳۰۰ پیکسل و حداکثر یک سهم مساوی (`1fr`) باشد. به تعداد ستونی که در عرض صفحه جا می‌شود بساز».

```
صفحه‌ی ۴  ←  ۱۲۰۰px ستون
صفحه‌ی ۲  ←  ۷۰۰px ستون
صفحه‌ی ۱  ←  ۳۵۰px ستون
```

بدون media query، چیدمان خودکار ریسپانسیو می‌شود. اگر کارت‌ها بزرگ به نظر می‌رسند، عدد `300px` را کمتر کن.

۶. واحدها

واحد	معنی
`px`	پیکسل ثابت
`rem`	نسبت به اندازه‌ی فونت ریشه (معمولاً ۱۶px)؛ برای فاصله و فونت بهتر از px است
`%`	درصد از والد
`vh`	درصدی از ارتفاع صفحه‌ی نمایش
`fr`	سهم از فضای باقی‌مانده در Grid

۷. aspect-ratio و object-fit

```
.movie-poster { aspect-ratio: 2/3; width: 100%; }
.movie-poster img { width: 100%; height: 100%; object-fit: cover; }
```

- `aspect-ratio: 2/3` : نسبت پوستر را ثابت نگه می‌دارد (عرض ۲، ارتفاع ۳).
- `object-fit: cover` : عکس کادر را کاملاً پر می‌کند و اگر لازم شد از لبه‌ها برش می‌خورد، ولی کشیده نمی‌شود.

۸. position: relative و absolute (لایه‌ی روی پوستر)

```
.movie-poster  { position: relative; }
.movie-overlay { position: absolute; top:0; left:0; right:0; bottom:0; opacity: 0; }
.movie-card:hover .movie-overlay { opacity: 1; }
```

- والد `relative` است تا فرزند `absolute` نسبت به **او** جایگذاری شود.
- `top/left/right/bottom: 0` یعنی لایه تمام پوستر را بپوشاند.
- `opacity: 0` ابتدا نامرئی است و با `:hover` روی کارت، `1` می‌شود. دکمه‌ی قلب روی عکس هم از همین لایه است.

۹. transition، transform و animation

```
.movie-card { transition: transform 0.2s; }
.movie-card:hover { transform: translateY(-5px); }
```

`transition` تغییر را نرم می‌کند، و `transform: translateY(-5px)` کارت را ۵ پیکسل بالا می‌برد. برای انیمیشن‌های پیچیده‌تر:

```
@keyframes fadeIn { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }
.movies-grid > * { animation: fadeIn 0.3s ease-out forwards; }
```

۱۰. انتخابگرها

انتخابگر	معنی
`.a`	عنصری با کلاس `a`
`.a .b`	`b` که **داخل** `a` است (هر عمقی)
`.a > *`	فرزندان **مستقیم** `a`
`.a:hover`	وقتی ماوس روی `a` است
`.a h2`	هر `h2` داخل `a`

۱۱. media query (ریسپانسیو)

```
@media (max-width: 768px) {
  .navbar { padding: 1rem; }
}
```

«فقط وقتی عرض صفحه حداکثر ۷۶۸ پیکسل بود (موبایل و تبلت)، این قانون‌ها اعمال شوند».

۱۲. متغیرها و حالت‌ها

در `index.css`، `:root` فونت و رنگ پایه را می‌گذارد و `prefers-color-scheme: light` برای حالت روشن سیستم، رنگ‌ها را عوض می‌کند.

بخش ۷: منطق کل برنامه به زبان الگوریتم

الف) باز شدن سایت

```
۱. مرورگر index.html را می‌گیرد و main.jsx را اجرا می‌کند
۲. React برنامه را در div#root می‌سازد
۳. Router آدرس را می‌خواند و صفحه‌ی مربوط را انتخاب می‌کند
۴. اگر صفحه Home است:
      نشان بده "Loading" ← loading=true...
      بعد از نمایش، لیست فیلم‌های محبوب را درخواست کن
      اگر پاسخ آمد: movies را پر کن، loading=false
      اگر خطا شد: پیام خطا بگذار، loading=false
```

ب) جستجو

```
۱. کاربر تایپ می‌کند ← با هر حرف searchQuery به‌روز می‌شود
۲. کاربر Enter یا دکمه‌ی search را می‌زند
۳. handleSearch اجرا می‌شود:
      اگر متن خالی است ← خروج
      اگر در حال بارگذاری است ← خروج
      loading=true
      نتیجه را از API بگیر
      movies = نتیجه  و  error = null
      (در خطا) error = "Failed to search movies"...
      loading=false
۴. React صفحه را با فیلم‌های جدید می‌سازد
```

ج) ناوبری

```
۱. کاربر روی Favorites کلیک می‌کند
۲. Link جلوی reload را می‌گیرد و آدرس را به /favorites/ تغییر می‌دهد
۳. Routes آدرس جدید را می‌بیند و Favorites را نشان می‌دهد
۴. NavBar تغییر نمی‌کند (چون بیرون Routes است)
۵. Home از صفحه حذف می‌شود (state آن از بین می‌رود)
```

نکته: وقتی از Home به Favorites و برگشت به Home می‌روی، چون `Home` از بین رفته بود، دوباره **از نو ساخته می‌شود** و دوباره فیلم‌های محبوب را می‌گیرد، و متن جستجو پاک می‌شود. برای نگه داشتن این‌ها باید state را بالاتر ببری (Context).

د) ماشین حالت (State Machine) صفحه‌ی Home

loading	error	movies	صفحه چه نشان می‌دهد
true	null	[]	«Loading...»
false	null	پر	شبکه‌ی فیلم‌ها

loading	error	movies	صفحه چه نشان می‌دهد
false	پیام	[]	پیام خطا (و شبکه‌ی خالی)
false	null	[]	شبکه‌ی خالی (جستجوی بی‌نتیجه)

حالت چهارم یک **حالت خالی** است که الان پیامی ندارد؛ بهتر است «No movies found» نشان داده شود.

بخش ۸: مرور کد (باگ‌ها و بهبودها)

این بخش مهم‌ترین قسمت برای review است. موارد را به ترتیب اولویت ببین.

۱. باگ: غلط املایی `satError` (Home.jsx، تابع handleSearch)

```
} catch (err) {
  console.log(err);
  satError("Failed to search movies...");   // باید setError باشد
}
```

`satError` تعریف نشده است. اگر جستجو خطا بدهد، به جای نمایش پیام، خود `catch` خطای `ReferenceError` پرتاب می‌کند. (`finally` باز هم اجرا می‌شود و loading خاموش می‌شود، ولی کاربر پیامی نمی‌بیند و در کنسول یک خطای unhandled می‌آید.)

درس: غلط‌های املایی در اسم‌ها شایع‌ترین دلیل خطاست. از `F2` برای تغییر اسم و از lint استفاده کن.

۲. خطر: بررسی نشدن `response.ok` (api.js)

`fetch` برای پاسخ‌های `401` یا `500` خطا پرتاب **نمی‌کند**. در این حالت `response.json()` یک آبجکت خطا می‌دهد که `results` ندارد و `data.results` می‌شود `undefined`. بعد `setMovies(undefined)` و در رندر `movies.map` خطا می‌دهد و **صفحه سفید می‌شود**.

```
if (!response.ok) throw new Error("Request failed: " + response.status);
```

با این خط، خطا به `catch` در `Home` می‌رسد و پیام مناسب نمایش داده می‌شود.

۳. امنیت: کلید API داخل کد

`API_Key` مستقیم در فایل است. اگر پروژه را روی GitHub عمومی بگذاری، کلیدت لو می‌رود. به `.env` ببر:

```
VITE_TMDB_KEY=کلید_تو
```

و در کد: `import.meta.env.VITE_TMDB_KEY`. و `.env` را در `.gitignore` بگذار. (به یاد داشته باش هر چیز فرانت‌اند در نهایت برای کاربر قابل دیدن است، پس برای کلیدهای واقعاً محرمانه به بک‌اند نیاز داری.)

۴. تجربه‌ی کاربری هنگام جستجو

وقتی `loading` درست است، **کل شبکه‌ی فیلم‌ها** با «Loading...» عوض می‌شود. اگر بخواهی نرم‌تر شود، می‌توانی فقط یک نشانگر کوچک نشان بدهی. این یک انتخاب طراحی است، نه باگ.

۵. حالت خالی نیست

اگر جستجو هیچ نتیجه‌ای نداشته باشد، کاربر یک صفحه‌ی خالی می‌بیند. پیشنهاد:

```
{!loading && !error && movies.length === 0 && <p>No movies found</p>}
```

۶. پوستر `null`

برای فیلمی که `poster_path` ندارد، آدرس عکس خراب می‌شود. راه‌حل: شرط یا عکس جایگزین.

۷. `error` در جستجوی موفق پاک می‌شود ولی در بارگذاری اولیه نه

در `handleSearch` بعد از موفقیت `setError(null)` هست ولی در ابتدای `handleSearch` پاک نمی‌شود؛ بهتر است قبل از درخواست `setError(null)` بزنی تا پیام قبلی هم‌زمان با loading باقی نماند.

۸. دکمه‌ی علاقه‌مندی هنوز `alert` است

بخش اصلی پروژه که باید با Context و `localStorage` کامل شود (در مراحل بعد).

۹. نام‌گذاری و حروف

- فایل `NavBar.jsx` ولی CSS با نام `Navbar.css` ایمپورت شده؛ در ویندوز مشکلی نیست ولی در Linux (Deploy) اگر حروف فرق کند خطا می‌دهد. اسم‌ها را دقیق یکی کن.
- متغیر `API_Key` را طبق قرارداد ثابت‌ها `API_KEY` بنویس.
- `console.log(err)` برای خطاها بهتر است `console.error(err)` باشد.

۱۰. جزئیات ظاهری کد

- فرمت `main.jsx` و `Home.jsx` (تو رفتگی و `catch` زیر `}`) ناهمگن است. Prettier را نصب کن تا یکدست شود.
- کامنت `// movie.title.toLowerCase().startsWith(searchQuery)` داخل JSX باقی مانده. حذفش کن.
- `import React` در `NavBar.jsx` اضافه است.

فهرست کوتاه اولویت‌ها

اولویت	مورد
بالا	درست کردن `satError`
بالا	بررسی `response.ok` در `api.js`
بالا	انتقال کلید API به `.env`
متوسط	پیام «نتیجه‌ای پیدا نشد»
متوسط	مدیریت `poster_path` خالی
متوسط	پیاده‌سازی علاقه‌مندی‌ها با Context
کم	Prettier، نام‌گذاری، پاک‌سازی کامنت‌ها

بخش ۹: سؤال‌های مرور با پاسخ

۱. چرا `div#root` در `index.html` خالی است؟

چون React محتوای برنامه را با جاوااسکریپت داخل همین ظرف می‌سازد.

۲. تفاوت `Link` و `a` چیست؟

`a` صفحه را کامل از سرور دوباره بارگیری می‌کند ولی `Link` فقط آدرس را عوض می‌کند و React محتوای مناسب را نشان می‌دهد (بدون reload).

۳. چرا `BrowserRouter` باید بیرون `App` باشد؟

چون `Routes` و `Link` داخل `App` برای کار کردن باید داخل یک Router باشند تا بدانند آدرس فعلی چیست.

۴. state چیست و تفاوتش با props؟

state حافظه‌ی خود کامپوننت است و با تغییرش، رندر مجدد رخ می‌دهد. props ورودی از والد است و فقط خواندنی.

۵. چرا `movies` با `useState([])` شروع می‌شود، نه `null`؟

چون `movies.map` روی `[]` کار می‌کند (هیچ چیز نمایش نمی‌دهد) ولی روی `null` خطا می‌دهد.

۶. در `useEffect` آرایه‌ی `[]` چه می‌کند؟

باعث می‌شود effect فقط یک بار بعد از اولین نمایش اجرا شود.

۷. چرا `setLoading(false)` در `finally` است؟

چون چه درخواست موفق باشد چه ناموفق، باید loading خاموش شود. در غیر این صورت در خطا «Loading...» برای همیشه می‌ماند.

۸. چرا `e.preventDefault()`؟

چون مرورگر پس از ارسال فرم صفحه را reload می‌کند و state از دست می‌رود.

۹. `key` برای چیست؟

برای اینکه React هنگام تغییر لیست بتواند هر آیتم را شناسایی و تغییرات را بهینه اعمال کند.

۱۰. چرا `movie.release_date?.split("-")[0]` سال را می‌دهد؟

`split("-")` تاریخ `2020-10-22` را به `["2020","10","22"]` تبدیل می‌کند و `[0]` اولین قسمت را برمی‌دارد. `?.` از خطا هنگام نبودن تاریخ جلوگیری می‌کند.

۱۱. `encodeURIComponent` برای چیست؟

تا کاراکترهای خاص متن جستجو (مثل `&` و فاصله) آدرس را خراب نکنند.

۱۲. اگر `fetch` پاسخ ۴۰۱ بدهد چه می‌شود و چرا؟

`fetch` خطا پرتاب نمی‌کند؛ `response.json()` آبجکت خطا می‌دهد، `data.results` تعریف نشده است و `movies.map` ممکن است صفحه را از کار بیندازد. باید `response.ok` را چک کرد.

۱۳. چرا کارت‌ها اول بزرگ شدند؟

چون `className` در JSX `movie-grid` بود ولی CSS برای `.movies-grid` نوشته شده بود؛ پس grid اعمال نشد.

۱۴. `grid-template-columns: repeat(auto-fit, minmax(300px, 1fr))` چه می‌کند؟

به تعداد ستونی که در عرض صفحه جا می‌شود (هر ستون حداقل ۳۰۰px) ستون می‌سازد و فضای باقی‌مانده را مساوی تقسیم می‌کند.

۱۵. چرا اگر state را مستقیم عوض کنیم کار نمی‌کند؟

چون React فقط وقتی از تغییر مطلع می‌شود که تابع set صدا زده شود؛ تغییر مستقیم رندر مجدد را راه نمی‌اندازد.

۱۶. چرا در StrictMode درخواست API دو بار می‌رود؟

چون در حالت توسعه، React عمداً effect را دو بار اجرا می‌کند تا مشکلات پنهان را نشان بدهد. در build نهایی اتفاق نمی‌افتد.

۱۷. چرا NavBar بیرون Routes است؟

تا در همه‌ی صفحه‌ها ثابت بماند و فقط محتوای `main` عوض شود.

۱۸. چرا کد fetch در `api.js` است و نه در `Home.jsx` ؟

برای جداسازی مسئولیت‌ها، قابلیت استفاده‌ی مجدد، و تغییر راحت‌تر وقتی API عوض شود.

بخش ۱۰: واژه‌نامه

اصطلاح	معنی
Component	تابعی که JSX برمی‌گرداند؛ تکه‌ی UI
JSX	نوشتن شبیه HTML داخل جاوااسکریپت
Props	ورودی‌های کامپوننت از والد
State	حافظه‌ی کامپوننت که تغییرش رندر مجدد می‌سازد
Render	ساختن JSX بر اساس state و props
Hook	تابع ویژه‌ی React (مثل `useState`) که با `use` شروع می‌شود
Effect	کار جانبی بعد از رندر (مثل fetch)
Dependency array	فهرست مقادیری که تعیین می‌کنند effect کی اجرا شود
Controlled input	input که مقدارش از state می‌آید
Router	سیستم نمایش صفحه‌ی مناسب بر اساس آدرس
SPA	برنامه‌ی تک‌صفحه‌ای بدون بارگیری مجدد کامل
DOM	درخت عنصرهای صفحه در مرورگر
Virtual DOM	نسخه‌ی سبک DOM که React برای پیدا کردن تفاوت‌ها استفاده می‌کند
API	رابط ارتباط بین برنامه‌ها
Endpoint	یک آدرس مشخص از API
JSON	قالب متنی برای تبادل داده
Promise	قولی که بعداً با نتیجه پر می‌شود
async / await	روش نوشتن کد غیرهمزمان به شکل خطی
Optional chaining	`?.` برای جلوگیری از خطا روی مقدار تعریف‌نشده
Destructuring	بیرون کشیدن مقدارها از آبجکت یا آرایه
Immutability	تغییر ندادن داده‌ی قبلی و ساختن نسخه‌ی جدید
Guard clause	`return` زودهنگام برای شرط‌های نامعتبر
Empty state	نمایش مناسب وقتی داده‌ای نیست
Bundler	ابزار تبدیل و فشرده‌سازی کد (Vite)
Flexbox / Grid	سیستم‌های چیدمان CSS

اصطلاح	معنی
Media query	قانون CSS وابسته به اندازه‌ی صفحه

برای مرور نهایی: بدون نگاه به این سند، سعی کن جریان کامل «باز شدن سایت تا دیدن فیلم‌ها» را با زبان خودت توضیح بدهی، بعد سؤال‌های بخش ۹ را جواب بده. هر جا گیر کردی، به همان بخش برگرد.